

Pipeline in polynomial

```
In [18]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

```
In [21]: X=6*np.random.rand(100,1)-3
y=0.5*X**2+1.5*X+2+np.random.randn(100,1)
```

```
In [22]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_s
```

```
In [23]: from sklearn.pipeline import Pipeline

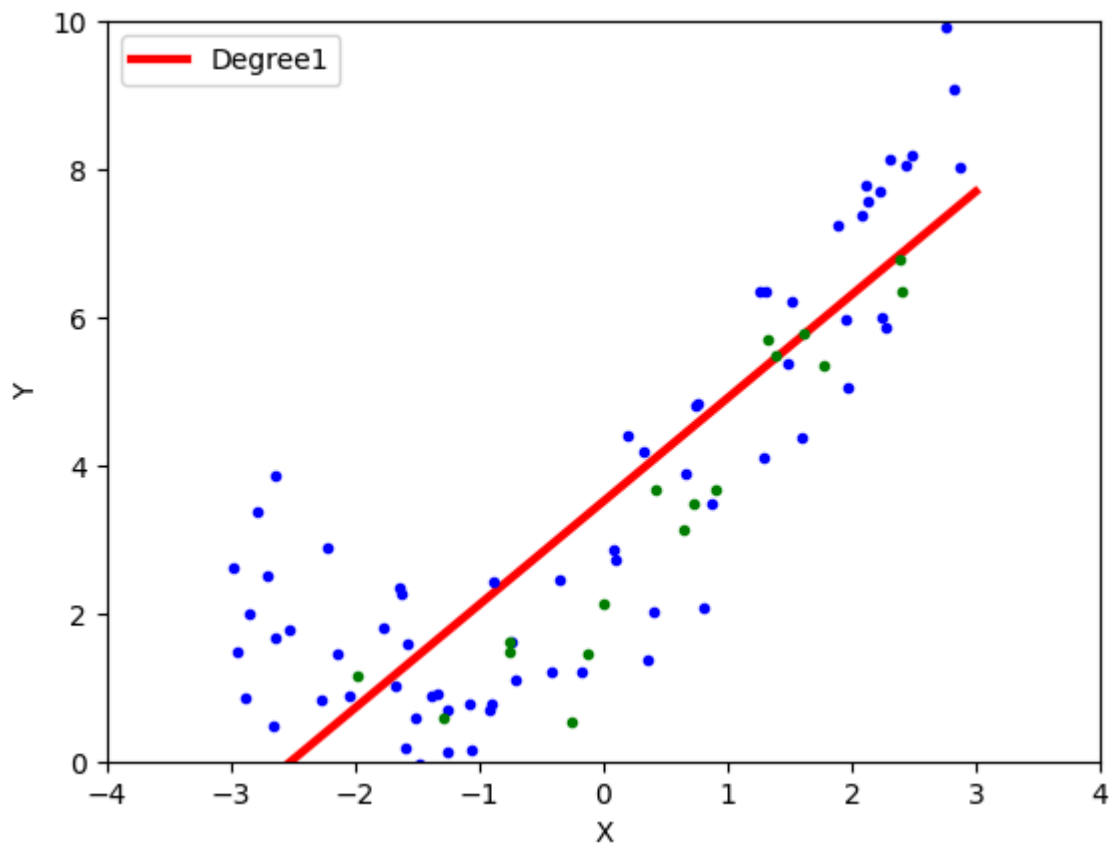
def poly_regression(degree):
    X_new=np.linspace(-3,3,200).reshape(200,1)
    poly_features=PolynomialFeatures(degree=degree,include_bias=True)
    lin_reg=LinearRegression()

    poly_regression=Pipeline([
        ('poly_features',poly_features),
        ('lin_reg',lin_reg)
    ])

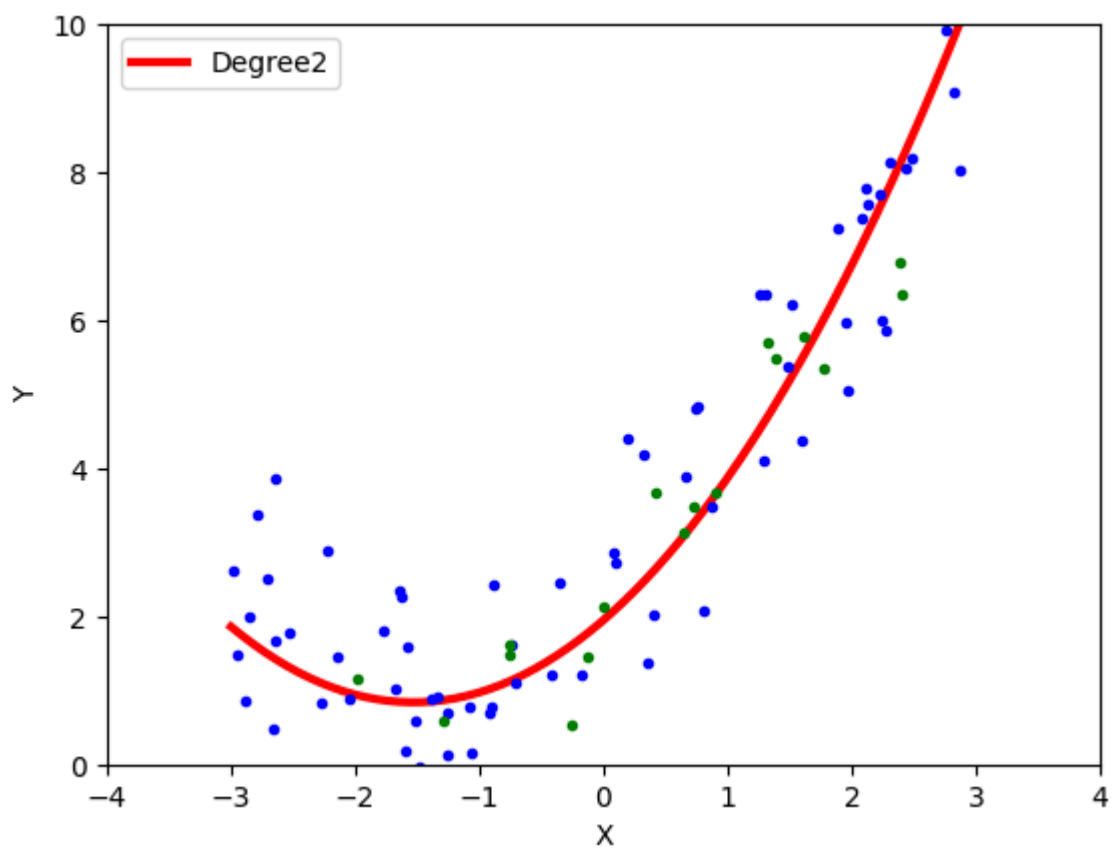
    poly_regression.fit(X_train,y_train)
    y_pred_new=poly_regression.predict(X_new)

    #plotting predicted lines
    plt.plot(X_new,y_pred_new,'r',label='Degree'+str(degree),linewidth=3)
    plt.plot(X_train,y_train,'b.',linewidth=3)
    plt.plot(X_test,y_test,'g.',linewidth=3)
    plt.legend(loc='upper left')
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.axis([-4,4,0,10])
    plt.show()
```

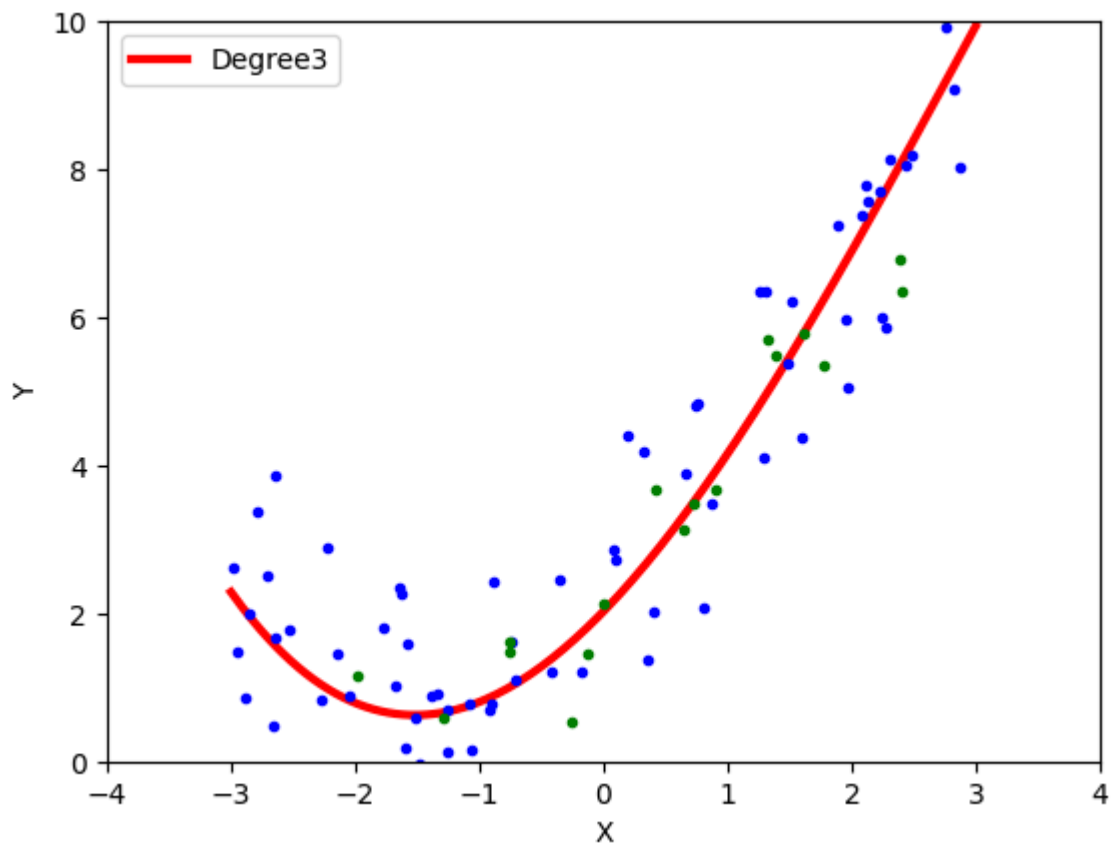
```
In [25]: poly_regression(1)
```



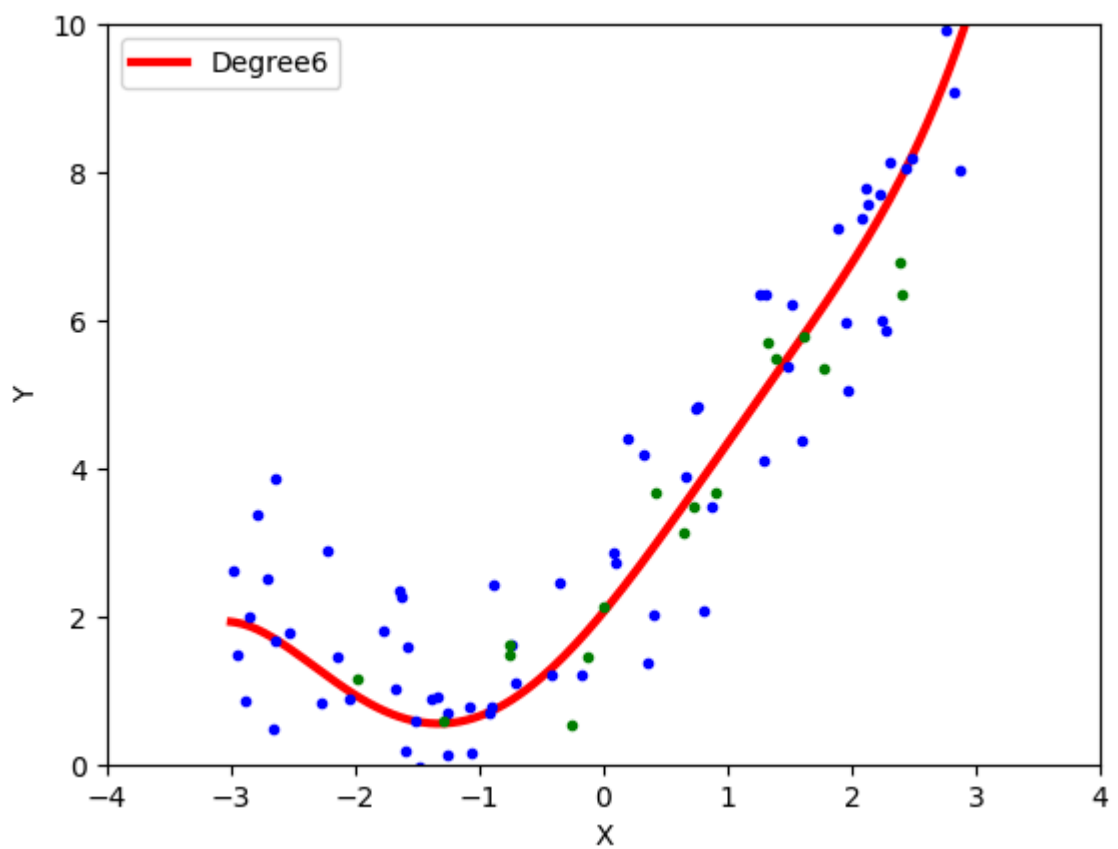
```
In [26]: poly_regression(2)
```



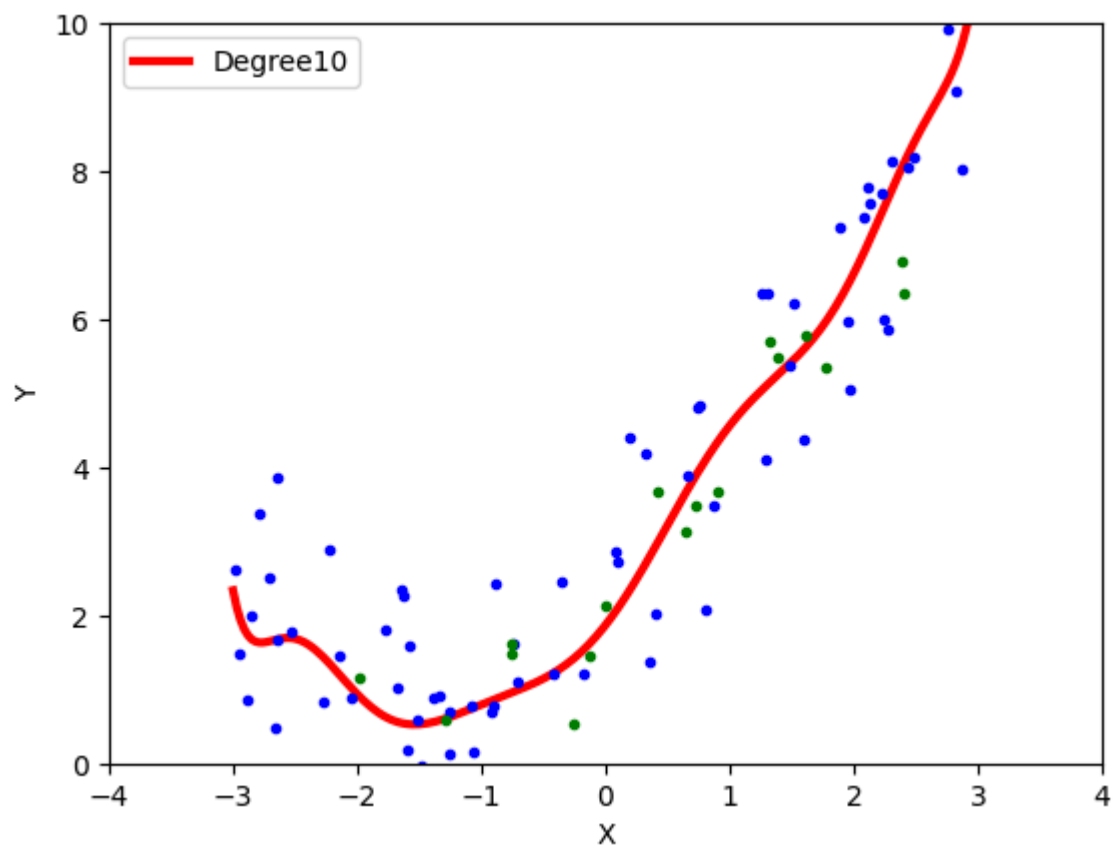
```
In [27]: poly_regression(3)
```



```
In [28]: poly_regression(6)
```



```
In [30]: poly_regression(10)  #overfitting
```



In []: